Send and receive messages from Azure Service Bus

In this exercise, you create and configure Azure Service Bus resources, then build a .NET app to send and receive messages using the **Azure.Messaging.ServiceBus** SDK. You learn how to provision a Service Bus namespace and queue, assign permissions, and interact with messages programmatically.

Tasks performed in this exercise:

- Create Azure Service Bus resources
- Assign a role to your Microsoft Entra user name
- Create a .NET console app to send and receive messages
- Clean up resources

This exercise takes approximately **30** minutes to complete.

Create Azure Event Hubs resources

In this section of the exercise you create the needed resources in Azure with the Azure CLI.

1. In your browser navigate to the Azure portal https*://portal.azure.com*; signing in with your Azure credentials if prompted.
2. Use the **[>_]** button to the right of the search bar at the top of the page to create a new cloud shell in the Azure portal, selecting a ***Bash*** environment. The cloud shell provides a command line interface in a pane at the bottom of the Azure portal. If you are prompted to select a storage account to persist your files, select **No storage account required**, your subscription, and then select **Apply**.

Note: If you have previously created a cloud shell that uses a *PowerShell* environment, switch it to ***Bash***.

3. In the cloud shell toolbar, in the **Settings** menu, select **Go to Classic version** (this is required to use the code editor).
4. Create a resource group for the resources needed for this exercise. If you already have a resource group you want to use, proceed to the next step. Replace **myResourceGroup** with a name you want to use for the resource group. You can replace **eastus** with a region near you if needed.

In this **Cloudslice** lab, your **Resource Group** has already been created (**myResourceGrouplod58047410**) and you may safely skip this step.

TypeCopy

```
az group create --name myResourceGroup --location eastus
```

5. Many of the commands require unique names and use the same parameters. Creating some variables will reduce the changes needed to the commands that create resources. Run the following commands to create the needed variables. Replace **myResourceGroup** with the name you're using for this exercise. If you changed the location in the previous step, make the same change in the **location** variable.

TypeCopy

resourceGroup=myResourceGrouplod58047410

location=eastus

namespaceName=svcbusns58047410

6. You will need the name assigned to the namespace later in this exercise. Run the following command and record output.

TypeCopy

echo $namespaceName

Create an Azure Service Bus namespace and queue

1. Create a Service Bus messaging namespace. The following command creates a namespace using the variable you created earlier. The operation takes a few minutes to complete.

bashTypeCopy

az servicebus namespace create \

--resource-group $resourceGroup \

--name $namespaceName \

--location $location

2. Now that a namespace is created, you need to create a queue to hold the messages. Run the following command to create a queue named **myqueue**.

bashTypeCopy

az servicebus queue create --resource-group $resourceGroup \

--namespace-name $namespaceName \

--name myqueue

Assign a role to your Microsoft Entra user name

To allow your app to send and receive messages, assign your Microsoft Entra user to the **Azure Service Bus Data Owner** role at the Service Bus namespace level. This gives your user account permission to manage and access queues and topics using Azure RBAC. Perform the following steps in the cloud shell.

1. Run the following command to retrieve the **userPrincipalName** from your account. This represents who the role will be assigned to.

TypeCopy

```
userPrincipal=$(az rest --method GET --url https://graph.microsoft.com/v1.0/me \
  --headers 'Content-Type=application/json' \
  --query userPrincipalName --output tsv)
```

2. Run the following command to retrieve the resource ID of the Service Bus namespace. The resource ID sets the scope for the role assignment to a specific namespace.

TypeCopy

```
resourceID=$(az servicebus namespace show --name $namespaceName \
  --resource-group $resourceGroup \
  --query id --output tsv)
```

3. Run the following command to create and assign the **Azure Service Bus Data Owner** role.

TypeCopy

```
az role assignment create --assignee $userPrincipal \
  --role "Azure Service Bus Data Owner" \
  --scope $resourceID
```

Create a .NET console app to send and receive messages

Now that the needed resources are deployed to Azure the next step is to set up the console application. The following steps are performed in the cloud shell.

Tip: Resize the cloud shell to display more information, and code, by dragging the top border. You can also use the minimize and maximize buttons to switch between the cloud shell and the main portal interface.

1. Run the following commands to create a directory to contain the project and change into the project directory.

TypeCopy

```
mkdir svcbus

cd svcbus
```

2. Create the .NET console application.

TypeCopy

```
dotnet new console
```

3. Run the following commands to add the **Azure.Messaging.ServiceBus** and **Azure.Identity** packages to the project.

TypeCopy

```
dotnet add package Azure.Messaging.ServiceBus

dotnet add package Azure.Identity
```

Add the starter code for the project

1. Run the following command in the cloud shell to begin editing the application.

TypeCopy

```
code Program.cs
```

2. Replace any existing contents with the following code. Be sure to review the comments in the code, and replace with the Service Bus namespace you recorded earlier.

csharpTypeCopy

```
using Azure.Messaging.ServiceBus;

using Azure.Identity;

using System.Timers;



// TODO: Replace <YOUR-NAMESPACE> with your Service Bus namespace

string svcbusNameSpace = "<YOUR-NAMESPACE>.servicebus.windows.net";

string queueName = "myQueue";
```

// ADD CODE TO CREATE A SERVICE BUS CLIENT

// ADD CODE TO SEND MESSAGES TO THE QUEUE

// ADD CODE TO PROCESS MESSAGES FROM THE QUEUE

// Dispose client after use

await client.DisposeAsync();

3. Press **ctrl+s** to save your changes.

Add code to send messages to queue

Now it's time to add code to create the Service Bus client and send a batch of messages to the queue.

1. Locate the **// ADD CODE TO CREATE A SERVICE BUS CLIENT** comment and add the following code directly after the comment. Be sure to review the code and comments.

csharpTypeCopy

// Create a DefaultAzureCredentialOptions object to configure the DefaultAzureCredential

DefaultAzureCredentialOptions options = new()

{

ExcludeEnvironmentCredential = true,

ExcludeManagedIdentityCredential = true

};

// Create a Service Bus client using the namespace and DefaultAzureCredential

// The DefaultAzureCredential will use the Azure CLI credentials, so ensure you are logged in

ServiceBusClient client = new(svcbusNameSpace, new DefaultAzureCredential(options));

2. Locate the **// ADD CODE TO SEND MESSAGES TO THE QUEUE** comment and add the following code directly after the comment. Be sure to review the code and comments.

csharpTypeCopy

// Create a sender for the specified queue

ServiceBusSender sender = client.CreateSender(queueName);

// create a batch

using ServiceBusMessageBatch messageBatch = await sender.CreateMessageBatchAsync();

// number of messages to be sent to the queue

const int numOfMessages = 3;

for (int i = 1; i <= numOfMessages; i++)

{

  // try adding a message to the batch

  if (!messageBatch.TryAddMessage(new ServiceBusMessage($"Message {i}")))

  {

    // if it is too large for the batch

    throw new Exception($"The message {i} is too large to fit in the batch.");

  }

}

```
try
{
    // Use the producer client to send the batch of messages to the Service Bus queue
    await sender.SendMessagesAsync(messageBatch);
    Console.WriteLine($"A batch of {numOfMessages} messages has been published to the queue.");
}
finally
{
    // Calling DisposeAsync on client types is required to ensure that network
    // resources and other unmanaged objects are properly cleaned up.
    await sender.DisposeAsync();
}

Console.WriteLine("Press any key to continue");
Console.ReadKey();
```

3. Press **ctrl+s** to save the file, then continue with the exercise.

Add code to process messages in the queue

1. Locate the **// ADD CODE TO PROCESS MESSAGES FROM THE QUEUE** comment and add the following code directly after the comment. Be sure to review the code and comments.

csharpTypeCopy

```
// Create a processor that we can use to process the messages in the queue
ServiceBusProcessor processor = client.CreateProcessor(queueName, new ServiceBusProcessorOptions());

// Idle timeout in milliseconds, the idle timer will stop the processor if there are no more
// messages in the queue to process
```

```
const int idleTimeoutMs = 3000;
System.Timers.Timer idleTimer = new(idleTimeoutMs);
idleTimer.Elapsed += async (s, e) =>
{
    Console.WriteLine($"No messages received for {idleTimeoutMs / 1000} seconds. Stopping processor...");
    await processor.StopProcessingAsync();
};

try
{
    // add handler to process messages
    processor.ProcessMessageAsync += MessageHandler;

    // add handler to process any errors
    processor.ProcessErrorAsync += ErrorHandler;

    // start processing
    idleTimer.Start();
    await processor.StartProcessingAsync();

    Console.WriteLine($"Processor started. Will stop after {idleTimeoutMs / 1000} seconds of inactivity.");
    // Wait for the processor to stop
    while (processor.IsProcessing)
    {
        await Task.Delay(500);
    }
    idleTimer.Stop();
```

```
    Console.WriteLine("Stopped receiving messages");
}
finally
{
    // Dispose processor after use
    await processor.DisposeAsync();
}

// handle received messages
async Task MessageHandler(ProcessMessageEventArgs args)
{
    string body = args.Message.Body.ToString();
    Console.WriteLine($"Received: {body}");

    // Reset the idle timer on each message
    idleTimer.Stop();
    idleTimer.Start();

    // complete the message. message is deleted from the queue.
    await args.CompleteMessageAsync(args.Message);
}

// handle any errors when receiving messages
Task ErrorHandler(ProcessErrorEventArgs args)
{
    Console.WriteLine(args.Exception.ToString());
    return Task.CompletedTask;
}
```

2. Press **ctrl+s** to save the file, then **ctrl+q** to exit the editor.

Sign into Azure and run the app

1. In the cloud shell command-line pane, enter the following command to sign into Azure.

TypeCopy

az login

You must sign into Azure - even though the cloud shell session is already authenticated.

Note: In most scenarios, just using *az login* will be sufficient. However, if you have subscriptions in multiple tenants, you may need to specify the tenant by using the --*tenant* parameter. See Sign into Azure interactively using Azure CLI for details.

2. Run the following command to start the console app. The app will pause a various stages and prompt you to press a key to continue. This gives you an opportunity to view the messages in the Azure portal.

TypeCopy

dotnet run

3. In the Azure portal, navigate to the Service Bus namespace you created.
4. Select **myqueue** at the bottom of the **Overview** window.
5. Select **Service Bus Explorer** in the left navigation pane.
6. Select **Peek from start** and the three messages should appear after a few seconds.
7. In the cloud shell, press any key to continue and the application will process the three messages.
8. Return to the portal after the application has completed processing the messages. Select **Peek from start** again and notice there are no messages in the queue.

Clean up resources

Now that you finished the exercise, you should delete the cloud resources you created to avoid unnecessary resource usage.

1. In your browser navigate to the Azure portal https:*//portal.azure.com*; signing in with your Azure credentials if prompted.

2. Navigate to the resource group you created and view the contents of the resources used in this exercise.
3. On the toolbar, select **Delete resource group**.
4. Enter the resource group name and confirm that you want to delete it.

CAUTION: Deleting a resource group deletes all resources contained within it. If you chose an existing resource group for this exercise, any existing resources outside the scope of this exercise will also be deleted.